

An exact null for the PCA/t-test unidimensionality protocol

Why the person-level t-test is not a 5% test, and what that does to the binomial step

Magnus Johansson

2026-09-16

Table of contents

Purpose	1
The statistic is the change index with subtests in place of occasions	1
Conditioning on the total score removes the latent distribution	2
Validation	4
The 1.96 rule is not a 5% test	4
A large share of respondents cannot be flagged at all	5
Asymmetry and centering	7
What happens at the count level	7
Extreme-score handling does not rescue it	9
What this changes about the protocol	10
Not yet done: the split is chosen from the data	11
Scope and limits	11
Reproduction	11
Bibliography	11

Purpose

The PCA/t-test protocol [1], [2] splits a scale into two item sets from a principal component analysis of residuals, estimates a person location from each set, compares them per respondent with a t-test against 1.96, and refers the resulting proportion of significant tests to a 5% benchmark through a binomial confidence interval. Hagell [3] showed that the conclusion depends on sample size and on which of five binomial interval methods is used, and argued the protocol should be read as ordinary hypothesis testing rather than as a definite test.

That leaves the reference proportion itself unexamined. This note shows that the 1.96 rule is not a 5% test, computes its exact size, and records what that does to the binomial step. The conclusion is that the choice of interval method is a second-order question about the shape of a distribution whose center is wrong by a factor of 1.3 to 3.7. The direction is uniform, so the protocol is biased toward declaring unidimensionality.

The statistic is the change index with subtests in place of occasions

The protocol computes

$$t = \frac{\hat{\theta}_A - \hat{\theta}_B}{\sqrt{SE(\hat{\theta}_A)^2 + SE(\hat{\theta}_B)^2}}$$

which is the Rasch change index of `dev/rci_exact_null.qmd` with item subtests in place of occasions. Both mechanisms documented there carry over.

Self-normalisation. $SE(\hat{\theta}_A)$ and $SE(\hat{\theta}_B)$ are deterministic functions of the realised subtest scores, so the denominator is not independent of the numerator, both tails of the null are damped, and the critical value falls below 1.96. The “t” label is a misnomer for the same reason: a t denominator is independent of its numerator, which is the one property this denominator does not have.

Discreteness. With complete data each subtest score is sufficient, so $\hat{\theta}_A$ and $\hat{\theta}_B$ take only as many values as there are subtest scores and the null is a distribution over $(k_A + 1) \times (k_B + 1)$ cells.

Conditional independence, which is what makes the enumeration possible, also carries over. In the change case the two occasions were independent given θ because they were separate administrations. Here the item sets are disjoint, so local independence gives $P(r_A, r_B | \theta) = P(r_A | \theta) P(r_B | \theta)$.

Conditioning on the total score removes the latent distribution

The RCI note enumerates the null at a set of person locations and pools over them, which makes the pooled critical value a property of the sample as well as the items. The subtest case admits a stronger construction. Under the Rasch model the conditional distribution of r_A given the total score $r_A + r_B$ is free of θ , by the same sufficiency that underlies CML estimation:

$$P(r_A = a | r_A + r_B = r) = \frac{\gamma_A(a) \gamma_B(r - a)}{\gamma(r)}$$

So the null of t can be enumerated per total score with no latent distribution assumption and no plug-in person estimate. Numerically it is easier to evaluate $P(r_A | \theta) P(r_B | \theta)$ at any convenient θ and normalise within each total score, which gives the same ratio. Invariance is checked below rather than assumed.

```
# Thresholds: k items, ncat response categories, locations spread over
# [-spread, spread]. Same constructor as dev/rci_exact_null.qmd.
mk <- function(k, ncat, spread = 1.2) {
  lapply(seq(-spread, spread, length.out = k),
    function(b) b + seq(-0.9, 0.9, length.out = ncat - 1L))
}

# Lord-Wingersky: exact sum-score distribution given theta under a PCM
score_dist <- function(thr_list, theta) {
  d <- 1
  for (thr in thr_list) {
    p <- easyRasch2:::pcm_cat_probs(theta, thr)
    nd <- numeric(length(d) + length(p) - 1L)
    for (a in seq_along(d)) nd[a + seq_along(p) - 1L] <-
      nd[a + seq_along(p) - 1L] + d[a] * p
    d <- nd
  }
  d
  # index = score + 1
}

# Score -> (theta, SE) lookup, WLE. The lookup RMscoreSE() builds.
lut <- function(thr_list) {
  steps <- vapply(thr_list, length, integer(1L))
  m <- vapply(0:sum(steps), function(r)
    easyRasch2:::theta_wle(easyRasch2:::score_pattern(r, steps),
      thr_list, c(-10, 10)), numeric(2L))
  list(theta = m[1L, ], se = m[2L, ])
}
```

```

# The t grid and the raw logit-gap grid for one split
split_grid <- function(thr_all, idxA) {
  A <- thr_all[idxA]; B <- thr_all[-idxA]
  lA <- lut(A); lB <- lut(B)
  list(A = A, B = B, lA = lA, lB = lB,
       t = outer(seq_along(lA$theta), seq_along(lB$theta),
                 function(i, j) (lA$theta[i] - lB$theta[j]) /
                               sqrt(lA$se[i]^2 + lB$se[j]^2)),
       d = outer(seq_along(lA$theta), seq_along(lB$theta),
                 function(i, j) lA$theta[i] - lB$theta[j]))
}

# Null of t conditional on the total score. Free of theta.
cond_null <- function(g, theta_eval = 0) {
  dA <- score_dist(g$A, theta_eval); dB <- score_dist(g$B, theta_eval)
  maxA <- length(dA) - 1L; maxB <- length(dB) - 1L
  lapply(0:(maxA + maxB), function(r) {
    a <- max(0L, r - maxB):min(maxA, r)
    w <- dA[a + 1L] * dB[r - a + 1L]
    list(a = a, p = w / sum(w),
         t = g$t[cbind(a + 1L, r - a + 1L)],
         d = g$d[cbind(a + 1L, r - a + 1L)])
  })
}

alpha_by_r <- function(cn, crit = 1.96)
  vapply(cn, function(x) sum(x$p[abs(x$t) > crit]), numeric(1L))

gap_by_r <- function(cn) vapply(cn, function(x) sum(x$p * x$d), numeric(1L))

# Total-score distribution under a latent density, by quadrature
score_weights <- function(thr_all, mu = 0, sigma = 1.4, nq = 121L) {
  q <- seq(mu - 5 * sigma, mu + 5 * sigma, length.out = nq)
  w <- dnorm(q, mu, sigma); w <- w / sum(w)
  Reduce('+', Map(function(t, wt) wt * score_dist(thr_all, t), q, w))
}

# Pooled exact critical values achieving nominal alpha
crit_pooled <- function(cn, fr, alpha = 0.05) {
  tv <- unlist(lapply(cn, `[[`, "t"))
  pv <- unlist(Map(function(x, f) x$p * f, cn, fr))
  o <- order(tv); tv <- tv[o]; pv <- pv[o] / sum(pv)
  cw <- cumsum(pv)
  c(lower = tv[which(cw >= alpha / 2)[1L]],
     upper = tv[which(cw >= 1 - alpha / 2)[1L]])
}

# Exact Poisson-binomial distribution of the flagged count
pois_binom <- function(p) {
  d <- 1
  for (pi in p) d <- c(d * (1 - pi), 0) + c(0, d * pi)
  d
  # index = count + 1
}

```

Validation

Three anchors, following the discipline of the RCI note: reproduce an identity to machine precision, and check the fast path against simulation.

```
thr <- mk(12, 4)
idxA <- seq(1, 12, by = 2)          # interleaved 6 / 6
g <- split_grid(thr, idxA)

# Anchor 1: is the conditional null actually free of theta?
cn0 <- cond_null(g, 0); cnm <- cond_null(g, -2); cnp <- cond_null(g, 2.5)
dev1 <- max(vapply(seq_along(cn0), function(i)
  max(abs(cn0[[i]]$p - cnm[[i]]$p), abs(cn0[[i]]$p - cnp[[i]]$p)), numeric(1L)))

# Anchor 3: does the exact pooled critical value recover nominal alpha?
fr <- score_weights(thr, 0, 1.4); fr <- fr / sum(fr)
cp <- crit_pooled(cn0, fr, 0.05)
ach <- sum(vapply(seq_along(cn0), function(i) fr[i] *
  sum(cn0[[i]]$p[cn0[[i]]$t < cp[["lower"]] | cn0[[i]]$t > cp[["upper"]]]),
  numeric(1L)))
```

Anchor 1, theta-invariance. Evaluating the conditional distribution at $\theta = -2, 0$ and 2.5 agrees to a maximum absolute difference of $1.9\text{e-}16$. The construction is free of θ rather than approximately so.

Anchor 2, against simulation. Four hundred thousand respondents on the 12-item scale, drawn from $N(0, 1.4)$, scored and classified by the same 1.96 rule. Per-total-score α agrees with the enumeration to a maximum absolute difference of 0.0040 against a Monte Carlo standard error of 0.0039. The simulation is not run at render time. It lives in `dev/ttest_protocol_exact_null.R`.

Anchor 3, self-consistency. Feeding the exact pooled critical values back through the enumeration gives $\alpha = 0.0492$ against a nominal 0.05, the residual being discreteness.

The 1.96 rule is not a 5% test

```
run <- function(label, k, ncat, idxA, mu = 0, sigma = 1.4) {
  thr <- mk(k, ncat); g <- split_grid(thr, idxA); cn <- cond_null(g, 0)
  fr <- score_weights(thr, mu, sigma); fr <- fr / sum(fr)
  a <- alpha_by_r(cn, 1.96); gp <- gap_by_r(cn); cp <- crit_pooled(cn, fr, 0.05)
  nz <- a > 0
  data.frame(config = label, subtests = paste0(length(idxA), "/", k - length(idxA)),
    alpha = sum(fr * a), ratio = 0.05 / sum(fr * a),
    pct_zero = 100 * sum(fr[!nz]),
    lo = cp[["lower"]], hi = cp[["upper"]],
    asym = abs(cp[["lower"]]) - cp[["upper"]],
    gap = max(abs(gp[nz])), stringsAsFactors = FALSE)
}

res <- do.call(rbind, list(
  run("12 items, 4 cat", 12, 4, seq(1, 12, by = 2)),
  run("12 items, 4 cat", 12, 4, seq(1, 12, by = 3)),
  run("20 items, 4 cat", 20, 4, seq(1, 20, by = 2)),
  run("20 items, 4 cat", 20, 4, seq(1, 20, by = 3)),
  run("20 items, 4 cat", 20, 4, seq(1, 20, by = 5)),
  run("30 items, 4 cat", 30, 4, seq(1, 30, by = 2)),
  run("40 items, 4 cat", 40, 4, seq(1, 40, by = 2)),
  run("20 items, 2 cat", 20, 2, seq(1, 20, by = 2)),
```

```

run("20 items, 7 cat",          20, 7, seq(1, 20, by = 2)),
run("20 items, 4 cat, easy/hard", 20, 4, 1:10),
run("20 items, 4 cat, 2 logits off", 20, 4, seq(1, 20, by = 2), mu = -2)))

kable(data.frame(res$config, res$subtests, round(res$alpha, 4),
               round(res$ratio, 2), round(res$pct_zero, 1)),
      col.names = c("Scale", "Split", "alpha", "Too lenient",
                    "Unflaggable (%)"),
      row.names = FALSE)

```

Scale	Split	alpha	Too lenient	Unflaggable (%)
12 items, 4 cat	6/6	0.0240	2.08	22.6
12 items, 4 cat	4/8	0.0231	2.17	12.8
20 items, 4 cat	10/10	0.0315	1.59	13.7
20 items, 4 cat	7/13	0.0312	1.60	7.0
20 items, 4 cat	4/16	0.0257	1.95	2.8
30 items, 4 cat	15/15	0.0355	1.41	7.3
40 items, 4 cat	20/20	0.0394	1.27	5.7
20 items, 2 cat	10/10	0.0137	3.65	44.2
20 items, 7 cat	10/10	0.0333	1.50	13.2
20 items, 4 cat, easy/hard	10/10	0.0328	1.53	3.6
20 items, 4 cat, 2 logits off	10/10	0.0177	2.83	45.7

The true size of the 1.96 rule runs from 1.4% to 3.9%. It never reaches 5%, and at 40 items split 20/20 it is still short by a factor of 1.27. Dichotomous items are the worst, with the polytomous-versus-dichotomous contrast at 0.0137, and a sample two logits off target is nearly as bad at 0.0177. Both findings match the RCI note, where response categories moved the critical value mainly in the step from two to more than two, and where mistargeting dropped the pooled value from 1.695 to 1.522 because a floor-bound sample is located where the statistic is most damped.

The effective test length here is the subtest, not the scale, which is what makes the protocol worse than the change case rather than equivalent to it. The RCI note's length table reads 1.644 at 4 items, 1.725 at 6, 1.815 at 10 and 1.868 at 20, four categories throughout. A 20-item scale splitting 10/10 is governed by two 10-item instruments. The published recommendation to use the protocol with at least 12 item response thresholds [2], [4] amounts to four items at four categories, which is the worst row of that table.

A large share of respondents cannot be flagged at all

The last column above counts respondents for whom **no attainable pair of subtest scores produces** $|t| > 1.96$. Their contribution to the significant count is zero with probability one, not 0.05. At 20 dichotomous items that is 44% of the sample and on an off-target polytomous scale 46%.

```

panel <- function(label, k, ncat, idxA) {
  g <- split_grid(mk(k, ncat), idxA)
  a <- alpha_by_r(cond_null(g, 0), 1.96)
  data.frame(label = label, r = seq_along(a) - 1L, alpha = a)
}
d_fig <- rbind(

```

```

panel("20 items, 4 cat, 10/10", 20, 4, seq(1, 20, by = 2)),
panel("12 items, 4 cat, 6/6", 12, 4, seq(1, 12, by = 2)),
panel("20 items, 2 cat, 10/10", 20, 2, seq(1, 20, by = 2))
d_fig$label <- factor(d_fig$label, levels = unique(d_fig$label))

ggplot(d_fig, aes(r, alpha)) +
  geom_hline(yintercept = 0.05, linetype = "dashed", colour = "grey40") +
  geom_step(direction = "mid") +
  facet_wrap(~ label, scales = "free_x") +
  labs(x = "Total score across all items", y = "Exact type I error of 1.96") +
  theme_bw() + er2_axis_margins()

```

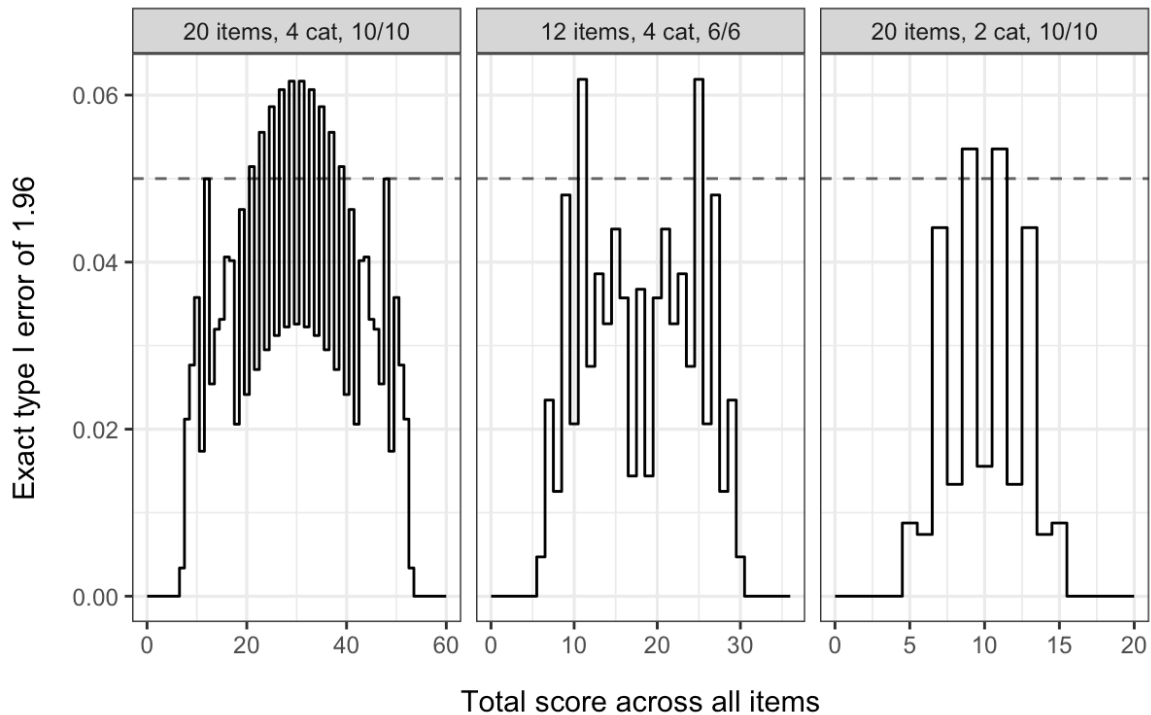


Figure 1: Exact size of the 1.96 rule by total score, under a perfectly unidimensional Rasch model. Weighted by a $N(0, 1.4)$ latent density to give the sample means quoted in the text. The dashed line is the 5% the protocol assumes. Scores at both ends carry a size of zero because no attainable split of that total clears 1.96.

The sawtooth is discreteness. Adjacent total scores differ in how many attainable splits they have and where those splits fall relative to the cutoff, so the size of the test alternates rather than varying smoothly. The RCI note found the same structure in the conditional critical value, which formed an arch across the score range rather than a constant.

The figure also shows that the conservatism is not a uniform shrinkage. Around the middle of the scale the size peaks at 0.062, 0.062 and 0.054 in the three panels, so the test is mildly anti-conservative there. Under the $N(0, 1.4)$ weighting, 20%, 6% and 10% of respondents hold a total score whose size exceeds 0.05. The whole of the deficit comes from the ends, where attainable scores run out and the size falls to zero. Averaging over the score distribution is what the protocol does when it pools respondents into one proportion, and that average is what has to be compared with 5%, but a reader should know that the pooled figure is combining two opposite behaviours rather than describing a test that is uniformly too strict.

Asymmetry and centering

```
sel <- c(1, 2, 5, 10, 3)
kable(data.frame(res$config[sel], res$subtests[sel], round(res$lo[sel], 3),
  round(res$hi[sel], 3), round(res$asym[sel], 3),
  round(res$gap[sel], 3)),
  col.names = c("Scale", "Split", "Lower", "Upper", "Asym.",
    "Gap (logits)"),
  row.names = FALSE)
```

Scale	Split	Lower	Upper	Asym.	Gap (logits)
12 items, 4 cat	6/6	-1.711	1.672	0.040	0.002
12 items, 4 cat	4/8	-1.767	1.626	0.140	0.114
20 items, 4 cat	4/16	-1.770	1.680	0.090	0.557
20 items, 4 cat, easy/hard	10/10	-2.001	1.578	0.423	0.195
20 items, 4 cat	10/10	-1.806	1.813	-0.007	0.001

The RCI note corrected an earlier belief that two-sided simulated critical values were asymmetric: under the change null the two occasions are exchangeable, so the difference is symmetric about zero however skewed $\hat{\theta}$ is, and exchangeability fails only when the occasions differ in missingness. Subtests break exchangeability by construction, because they differ in length, targeting and information. A balanced interleaved split is nearly symmetric, so the change result transfers there. An unbalanced or difficulty-confounded split is not. The ten easiest items against the ten hardest gives correct two-sided values of -2.001 and 1.578, so the upper value is stricter than 1.96 while the lower is more lenient. A principal component of residuals often separates items by location as well as by content, which makes that row the relevant one.

The last column is $\max_r |E[\hat{\theta}_A - \hat{\theta}_B | r]|$ under a perfectly unidimensional model. On a 4/16 split it reaches 0.557 logits, purely from the difference in WLE bias between a four-item and a sixteen-item subtest. That is model-implied signal with no multidimensionality behind it. Reporting quantity and units at every surface, as the RCI note insists, matters here: the gap is in logits, the t is unitless, and only the second is being compared with 1.96.

What happens at the count level

```
count_null <- function(label, k, ncat, idxA, n, mu = 0, sigma = 1.4) {
  thr <- mk(k, ncat); g <- split_grid(thr, idxA); cn <- cond_null(g, 0)
  fr <- score_weights(thr, mu, sigma); fr <- fr / sum(fr)
  a <- alpha_by_r(cn, 1.96)
  nr <- round(n * fr); p_i <- rep(a, times = nr)
  d <- pois_binom(p_i); cnt <- seq_along(d) - 1L; m <- length(p_i)
  mean_p <- sum(d * cnt) / m
  data.frame(config = label, n = m,
    mean_p = 100 * mean_p,
    q95 = 100 * cnt[which(cumsum(d) >= 0.95)[1L]] / m,
    p_over = sum(d[cnt / m > 0.05]),
    sd_pb = sqrt(sum(d * cnt^2) - sum(d * cnt)^2),
    sd_bin = sqrt(m * mean_p * (1 - mean_p)),
    stringsAsFactors = FALSE)
}

cfigs <- list(list("20 items, 4 cat, 10/10", 20, 4, seq(1, 20, by = 2)),
```

```

list("12 items, 4 cat, 6/6", 12, 4, seq(1, 12, by = 2)),
list("20 items, 2 cat, 10/10", 20, 2, seq(1, 20, by = 2)),
list("40 items, 4 cat, 20/20", 40, 4, seq(1, 40, by = 2)))
cts <- do.call(rbind, lapply(cfgs, function(cf)
  do.call(rbind, lapply(c(200, 500, 1000), function(n)
    count_null(cf[[1]], cf[[2]], cf[[3]], cf[[4]], n))))))

kable(data.frame(cts$config, cts$n, round(cts$mean_p, 2), round(cts$q95, 2),
  round(cts$p_over, 4), round(cts$sd_pb, 2), round(cts$sd_bin, 2)),
  col.names = c("Scale", "n", "Mean (%)", "95th (%)", "P(>5%)",
    "SD PB", "SD bin"),
  row.names = FALSE)

```

Scale	n	Mean (%)	95th (%)	P(>5%)	SD PB	SD bin
20 items, 4 cat, 10/10	204	3.15	5.39	0.0586	2.48	2.49
20 items, 4 cat, 10/10	504	3.15	4.56	0.0104	3.90	3.92
20 items, 4 cat, 10/10	1002	3.15	4.09	0.0007	5.50	5.53
12 items, 4 cat, 6/6	201	2.39	4.48	0.0090	2.15	2.17
12 items, 4 cat, 6/6	498	2.40	3.61	0.0005	3.39	3.42
12 items, 4 cat, 6/6	1001	2.40	3.20	0.0000	4.81	4.84
20 items, 2 cat, 10/10	199	1.36	3.02	0.0003	1.61	1.63
20 items, 2 cat, 10/10	499	1.37	2.20	0.0000	2.57	2.60
20 items, 2 cat, 10/10	1000	1.37	2.00	0.0000	3.62	3.67
40 items, 4 cat, 20/20	210	3.96	6.19	0.2114	2.82	2.82
40 items, 4 cat, 20/20	507	3.96	5.52	0.1105	4.38	4.39
40 items, 4 cat, 20/20	1008	3.94	4.96	0.0439	6.16	6.18

At $n = 1000$, observing more than 5% significant tests is an event of probability 0.0007, 0.0000, 0.0000 and 0.044 under a perfectly unidimensional model. The protocol treats 5% as the boundary of acceptability. The exact null puts it in the upper tail.

The binomial family is fine. Conditioning on the observed scores makes the count Poisson-binomial rather than binomial, since each respondent carries their own α_r . The last two columns show the two standard deviations agree to within a percent throughout. The heterogeneity in α_r is too small relative to its mean to move the variance, because $\text{Var}_{\text{PB}} = n\bar{\alpha} - \sum \alpha_i^2$ differs from $n\bar{\alpha}(1 - \bar{\alpha})$ by $n \text{Var}(\alpha_i)$, which is order 10^{-4} against a mean of order 10^{-2} . Marginally, with person locations drawn iid from a latent density, the count is binomial in any case.

So the distributional shape Hagell [3] investigates is not where the protocol goes wrong. All five of the interval methods compared there are being applied to roughly the right distribution and compared against the wrong number.

```

dist_one <- function(label, k, ncat, idxA, n = 500) {
  thr <- mk(k, ncat); g <- split_grid(thr, idxA)
  fr <- score_weights(thr, 0, 1.4); fr <- fr / sum(fr)
  p_i <- rep(alpha_by_r(cond_null(g, 0), 1.96), times = round(n * fr))
  d <- pois_binom(p_i)
  data.frame(label = label, prop = 100 * (seq_along(d) - 1L) / length(p_i),
    dens = d)
}

```

```

}
d_cnt <- rbind(
  dist_one("20 items, 4 cat, 10/10", 20, 4, seq(1, 20, by = 2)),
  dist_one("12 items, 4 cat, 6/6", 12, 4, seq(1, 12, by = 2)),
  dist_one("20 items, 2 cat, 10/10", 20, 2, seq(1, 20, by = 2)))

ggplot(subset(d_cnt, prop <= 8), aes(prop, dens, colour = label)) +
  geom_vline(xintercept = 5, linetype = "dashed", colour = "grey40") +
  geom_step(direction = "mid") +
  labs(x = "Observed significant tests (%)", y = "Exact probability",
       colour = NULL) +
  theme_bw() + er2_axis_margins() +
  theme(legend.position = "bottom")

```

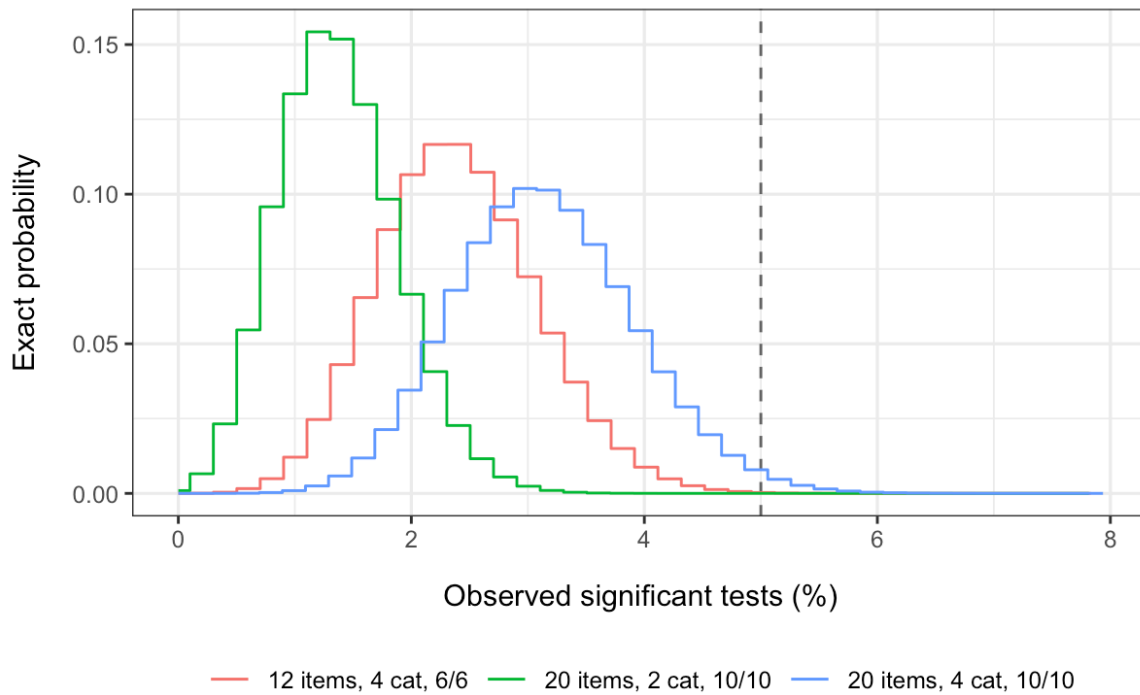


Figure 2: Exact null distribution of the observed proportion of significant t-tests at $n = 500$, under a perfectly unidimensional Rasch model with a $N(0, 1.4)$ latent density. The dashed line is the protocol's 5% benchmark.

Extreme-score handling does not rescue it

The lookup above uses WLE, which is finite at extreme scores. Software using maximum likelihood person estimates has to drop respondents extreme on either subtest, so the check is whether that changes the size.

```

joint_w <- function(thr_all, idxA, mu = 0, sigma = 1.4, nq = 121L) {
  A <- thr_all[idxA]; B <- thr_all[-idxA]
  q <- seq(mu - 5 * sigma, mu + 5 * sigma, length.out = nq)
  w <- dnorm(q, mu, sigma); w <- w / sum(w)
  Reduce('+', Map(function(t, wt) wt * outer(score_dist(A, t), score_dist(B, t)),
                q, w))
}

chk <- function(label, k, ncat, idxA, mu = 0) {
  thr <- mk(k, ncat); g <- split_grid(thr, idxA)

```

```

W <- joint_w(thr, idxA, mu); W <- W / sum(W)
sig <- abs(g$t) > 1.96
keep <- matrix(TRUE, nrow(W), ncol(W))
keep[c(1L, nrow(W)), ] <- FALSE; keep[, c(1L, ncol(W))] <- FALSE
data.frame(config = label, all = sum(W * sig),
            pct_ex = 100 * (1 - sum(W[keep])),
            dropped = sum(W[keep] * sig[keep]) / sum(W[keep]))
}
ex <- rbind(
  chk("12 items, 4 cat, 6/6",      12, 4, seq(1, 12, by = 2)),
  chk("20 items, 4 cat, 10/10",    20, 4, seq(1, 20, by = 2)),
  chk("20 items, 4 cat, 4/16",     20, 4, seq(1, 20, by = 5)),
  chk("40 items, 4 cat, 20/20",    40, 4, seq(1, 40, by = 2)),
  chk("20 items, 2 cat, 10/10",    20, 2, seq(1, 20, by = 2)),
  chk("20 items, 4 cat, off target", 20, 4, seq(1, 20, by = 2), mu = -2))
kable(data.frame(ex$config, round(ex$all, 4), round(ex$pct_ex, 1),
                round(ex$dropped, 4)),
       col.names = c("Scale", "alpha, all", "Extreme (%)",
                     "alpha, dropped"),
       row.names = FALSE)

```

Scale	alpha, all	Extreme (%)	alpha, dropped
12 items, 4 cat, 6/6	0.0240	8.4	0.0254
20 items, 4 cat, 10/10	0.0315	3.8	0.0323
20 items, 4 cat, 4/16	0.0257	9.8	0.0278
40 items, 4 cat, 20/20	0.0394	1.2	0.0397
20 items, 2 cat, 10/10	0.0137	22.0	0.0152
20 items, 4 cat, off target	0.0177	21.2	0.0218

Dropping extremes moves the size by two or three thousandths. The finding is not an artifact of the person estimator's behaviour at the boundary.

What this changes about the protocol

The protocol's decision rule compares an observed proportion against 5%. That benchmark is too lenient by a factor of 1.3 to 3.7 across everything tried here, worst on dichotomous items and off-target samples, which are common in health measurement. The direction is uniform, so the protocol is biased toward declaring unidimensionality.

This also gives a mechanism for Hagell's central finding [3]. Comparing against a benchmark that is too lenient shifts the whole power curve toward passing, so conclusions flipping with sample size is ordinary power behaviour against a biased reference rather than a pathology of the binomial interval. With the correct reference, mild multidimensionality would be rejected at appreciably smaller samples, and larger n would behave the way it does in any well posed test.

The constructive version is cheap. The enumeration needed to state the correct reference is the same machinery that produced it: one Lord-Wingersky recursion per subtest and one score-to-measure lookup, both of which the package already builds for `RMscoreSE()`. From that, either report the exact $\bar{\alpha}$ and test the observed count against it, or replace 1.96 with the exact per-score critical values so that the 5% benchmark becomes true by construction. The second is closer to what users expect the protocol to mean and needs no change to how results are reported.

Not yet done: the split is chosen from the data

Everything above conditions on a fixed, known split. The protocol chooses the split by principal component analysis of residuals, taking loadings above +0.3 and below -0.3, a cutoff whose rationale Hagell [3] already calls unclear. That is a selection step, and the null conditional on a data-chosen contrast is not the null for a fixed one. The enumeration therefore understates the size of the protocol as practised.

The two errors run in opposite directions. Referring to 1.96, together with the respondents who cannot be flagged at all, pushes the realised proportion below 5% and makes the protocol too lenient. Selecting the most discrepant contrast pushes it up and makes the protocol too strict. Which dominates depends on item count, response categories, targeting and how unbalanced the split is, and nothing here settles it.

Settling it needs a parametric bootstrap of the whole pipeline: generate under the fitted unidimensional model, run the PCA, take the split, run the t-tests, count. That is the same shape as `RMdimResidualPCACutoff()` and the infit and Q3 cutoff work, and the exact component above would serve as its reference, which removes most of the noise from the comparison the way the exact RCI null did for the change study.

It is worth holding open that the two errors partly cancel. Existing simulation work reporting the protocol as well behaved under unidimensionality would be consistent with that, and a cancellation would not be stable across the conditions in the first table. I have not checked what Smith [1] or Horton and Tennant [4] report on this point, so that reading is a conjecture to verify against the sources rather than a finding of this note.

Scope and limits

- **Complete data only**, as in the RCI note. Missing responses break sufficiency of a single subtest score, so the enumeration would run per missingness pattern.
- **Fixed split**. See the section above.
- **WLE with CML-style thresholds**. RUMM2030's person estimator may differ. It would move the centering column more than the size column, since dropping extremes barely changed the latter.
- **The latent density enters only through the weighting**. The per-score sizes are free of it. Only the sample means and the count distributions depend on the assumed $N(\mu, \sigma)$.
- The 1.96 figure is taken as the rule in force. Software referring to a t distribution with finite degrees of freedom uses a slightly larger value, which makes the test more conservative, not less.

Reproduction

`dev/ttest_protocol_exact_null.R` runs everything here plus the Monte Carlo anchor, which is not rendered. The enumeration is self-contained in the `exact-fns` chunk above and needs only `easyRasch2:::pcm_cat_probs()`, `:::theta_wle()` and `:::score_pattern()`, the same three internals the RCI note uses.

Bibliography

- [1] E. V. Smith Jr., "Detecting and Evaluating the Impact of Multidimensionality Using Item Fit Statistics and Principal Component Analysis of Residuals," *Journal of Applied Measurement*, vol. 3, pp. 205–231, 2002.
- [2] A. Tennant and J. Pallant, "Unidimensionality Matters," *Rasch Measurement Transactions*, vol. 20, pp. 1048–1051, 2006.
- [3] P. Hagell, "Testing Rating Scale Unidimensionality Using the Principal Component Analysis (PCA)/t-Test Protocol with the Rasch Model: The Primacy of Theory over Statistics," *Open Journal of Statistics*, vol. 4, pp. 456–465, 2014, doi: 10.4236/ojs.2014.46044.
- [4] M. Horton and A. Tennant, "Assessing Unidimensionality Using Smith's (2002) Approach in RUMM 2030," in *Probabilistic Models for Measurement in Education, Psychology, Social Science and Health*, Copenhagen, 2010.